

Posture Control

Through the human pose estimation model in the MediaPipe machine learning framework, the human body can be detected on the live camera feed. The joint-related positions of the human body can be marked. Then, various actions can be recognized in sequence, thereby achieving direct control of the robot through human body.

From the perspective of the robot, when you raise your left arm, the robot will move to the right; when you raise your right arm, the robot will move to the left; when your left leg is raised, the robot will move forward; and when your right leg is raised, the robot will move backward.

1. Program Logic

First, import the human pose estimation model from MediaPipe and subscribe to topic messages to obtain real-time footage from the camera.

Then, process the image, such as flipping, to detect human body information in the image. Based on the lines connecting the key points of the human body, calculate the limb height to determine the body movement.

Finally, if you make specified actions, the robot will perform the corresponding actions.

2. Operation Steps

Note: the input command should be case sensitive, and keywords can be complemented using Tab key.

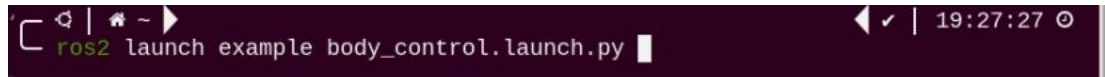
- 1) Start the robot, and enter the robot system desktop using VNC.
- 2) Click-on  to open the command line terminal.
- 3) Run the command to disable app auto-start app service.

```
~/stop_ros.sh
```

```
> ~/.stop_ros.sh
```

4) Run the following command and hit Enter key to initiate the game.

```
ros2 launch example body_control.launch.py
```

A terminal window with a dark background. The prompt is 'ros2' and the command 'launch example body_control.launch.py' is entered. The window title bar shows standard Linux window controls and a timestamp of 19:27:27.

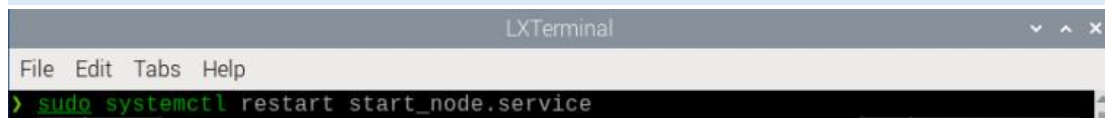
5) If you want to exit the game, press “Ctrl+C” in the terminal interface.

After experiencing the game, you can enable the app service through commands or by restarting the robot. If the app is not enabled, the related app functions will not work. If the robot is restarted, the app will be automatically enabled.



Click  and enter the command. Press enter to start the app, and wait for the buzzer to beep.

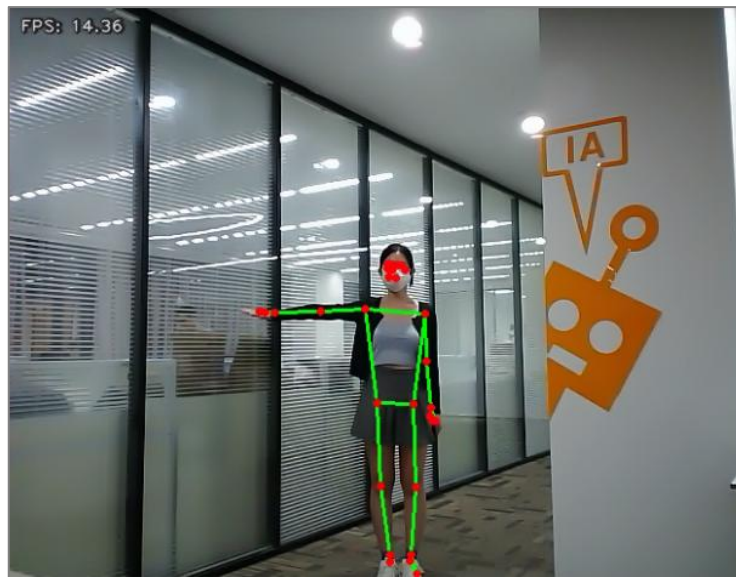
```
sudo systemctl restart start_node.service
```

A screenshot of an LXTerminal window. The title bar says 'LXTerminal'. The menu bar has 'File', 'Edit', 'Tabs', and 'Help'. The terminal text shows the command 'sudo systemctl restart start_node.service' being entered at the prompt.

3. Program Outcome

Once the game starts, ensure the human body remains as fully within the camera's field of view as possible. Upon recognizing the human body, the key points will be highlighted in the returned image.

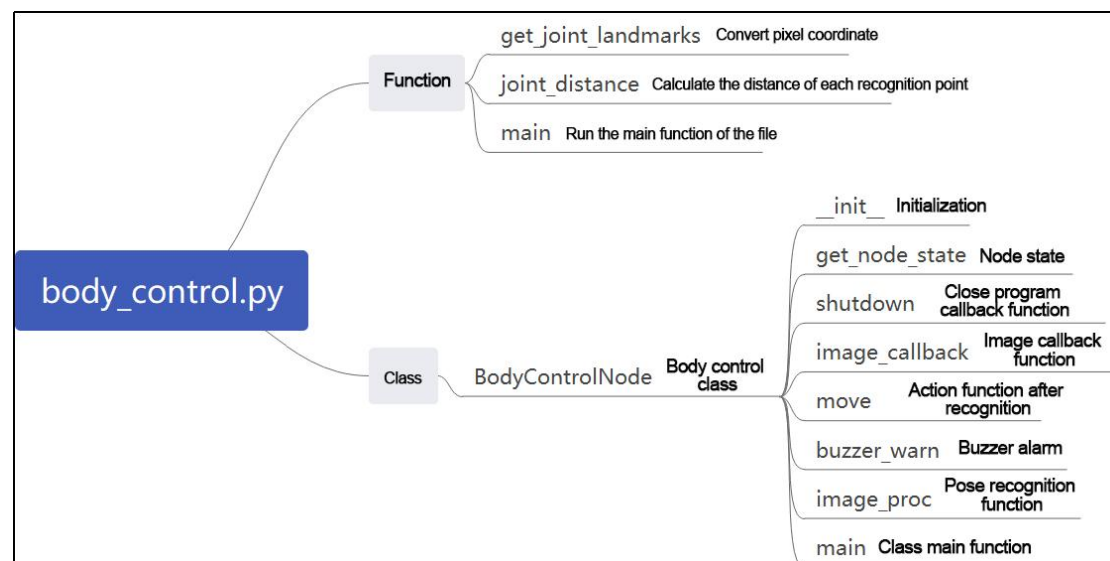
From the perspective of the robot, when you raises you left arm, the robot will turn left; when your right arm is raised, the robot will turn right; when your left leg is raised, the robot will move forward; and when your right leg is raised, the robot will move backward.



4. Program Analysis

The program file is saved in:

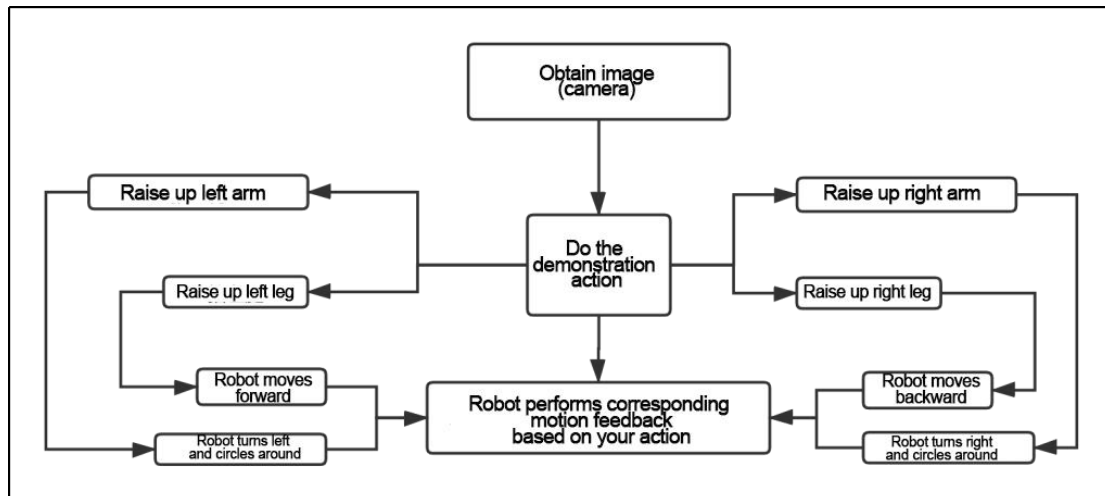
ros2_ws/src/example/example/body_control/include/body_control.py



Note: Prior to making any alterations to the program, ensure to create a backup of the original factory program. Modify it only after creating the backup. Directly editing the source code file is prohibited to prevent inadvertent parameter modifications that could render the robot dysfunctional and irreparable!

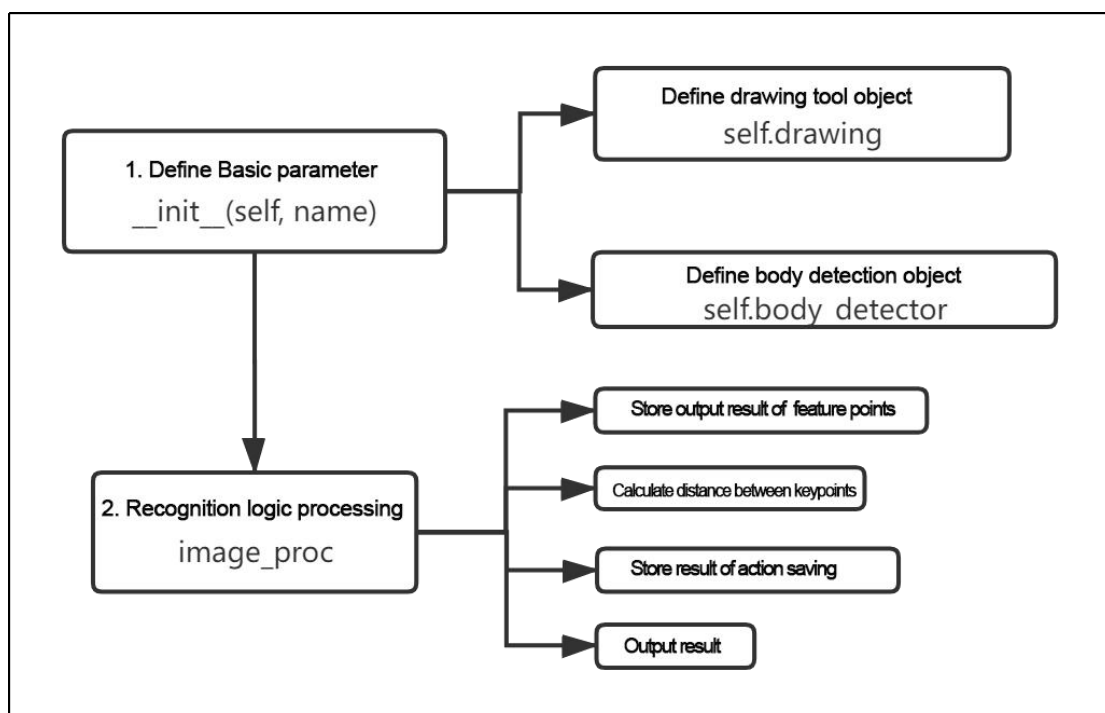
Based on the game's effectiveness, the procedural logic is delineated as

follows:



Obtaining the image through the camera, the robot will execute corresponding action after you make a pose. Taking the perspective of the robot, when you raise your left arm, the robot will turn left; when your right arm is raised, it will turn right; when your left leg is raised, it will move forward; and when your right leg is raised, it will move backward.

The program logic flow chart obtained from the program files is depicted in the figure below.



Define relevant parameters through the initialization function “init(self.name)”, including the image tool “self.drawing” object used to draw the recognized feature points, and the the body detection object “self.body_detector”. Then, perform the recognition logic processing for the output results of the recognized feature points. Judge the distance between the key points to save the actions. Finally, output the results to make the robot perform the corresponding actions.

4.1 Function

Main:

```
325 def main():
326     node = BodyControlNode('body_control')
327     rclpy.spin(node)
328     node.destroy_node()
```

Used to start the body sensation control node.

get_joint_landmarks:

```
44 def get_joint_landmarks(img, landmarks):
45     """
46     将landmarks从medipipe的归一化输出转为像素坐标(Convert landmarks from medipipe's normalized output to pixel coordinates)
47     :param img: 像素坐标对应的图片(picture corresponding to pixel coordinate)
48     :param landmarks: 归一化的关键点(normalized keypoint)
49     :return:
50     """
51     h, w, _ = img.shape
52     landmarks = [(lm.x * w, lm.y * h) for lm in landmarks]
53     return np.array(landmarks)
```

Used to convert the recognized information into pixel coordinates.

joint_distance:

```
58 def joint_distance(landmarks):
59     distance_list = []
60
61     d1 = landmarks[LEFT_HIP] - landmarks[LEFT_SHOULDER]
62     d2 = landmarks[LEFT_HIP] - landmarks[LEFT_WRIST]
63     dis1 = d1[0]**2 + d1[1]**2
64     dis2 = d2[0]**2 + d2[1]**2
65     distance_list.append(round(dis1/dis2, 1))
66
67     d1 = landmarks[RIGHT_HIP] - landmarks[RIGHT_SHOULDER]
68     d2 = landmarks[RIGHT_HIP] - landmarks[RIGHT_WRIST]
69     dis1 = d1[0]**2 + d1[1]**2
70     dis2 = d2[0]**2 + d2[1]**2
71     distance_list.append(round(dis1/dis2, 1))
72
73     d1 = landmarks[LEFT_HIP] - landmarks[LEFT_ANKLE]
74     d2 = landmarks[LEFT_ANKLE] - landmarks[LEFT_KNEE]
75     dis1 = d1[0]**2 + d1[1]**2
76     dis2 = d2[0]**2 + d2[1]**2
77     distance_list.append(round(dis1/dis2, 1))
78
79     d1 = landmarks[RIGHT_HIP] - landmarks[RIGHT_ANKLE]
80     d2 = landmarks[RIGHT_ANKLE] - landmarks[RIGHT_KNEE]
81     dis1 = d1[0]**2 + d1[1]**2
82     dis2 = d2[0]**2 + d2[1]**2
83     distance_list.append(round(dis1/dis2, 1))
```

Used to calculate the distance between each key point based on the pixel coordinate.

4.2 Class

```

84 class BodyControlNode(Node):
85     def __init__(self, name):
86         rclpy.init()
87         super().__init__(name, allow undeclared_parameters=True, automatically_declare_parameters_from_overrides=True)
88         self.name = name
89         self.drawing = mp.solutions.drawing_utils
90         self.body_detector = mp_pose.Pose(
91             static_image_mode=False,
92             min_tracking_confidence=0.7,
93             min_detection_confidence=0.7)
94         self.running = True
95         self.fps = fps.FPS() # fps计算器(FPS calculator)
96         signal.signal(signal.SIGINT, self.shutdown)
97
98         self.move_finish = True
99         self.stop_flag = False
100         self.left_hand_count = []
101         self.right_hand_count = []
102         self.left_leg_count = []
103         self.right_leg_count = []
104
105         self.detect_status = [0, 0, 0, 0]

```

This class is the body control node.

Init:

```

85     def __init__(self, name):
86         rclpy.init()
87         super().__init__(name, allow undeclared_parameters=True, automatically_declare_parameters_from_overrides=True)
88         self.name = name
89         self.drawing = mp.solutions.drawing_utils
90         self.body_detector = mp_pose.Pose(
91             static_image_mode=False,
92             min_tracking_confidence=0.7,
93             min_detection_confidence=0.7)
94         self.running = True
95         self.fps = fps.FPS() # fps计算器(FPS calculator)
96         signal.signal(signal.SIGINT, self.shutdown)

```

Initialize the parameters required for body control, read the camera's image callback node, initialize nodes such as servo, chassis, buzzer, motor, and others, and finally start the main function within the class.

get_node_state:

```

134     def get_node_state(self, request, response):
135         response.success = True
136         return response

```

Set the initialization state of the current node.

shutdown:

```

138     def shutdown(self, signum, frame):
139         self.running = False

```

Program exit callback function used to terminate recognition.

image_callback:


```

133 def image_callback(self, ros_image):
134     cv_image = self.bridge.imgmsg_to_cv2(ros_image, "rgb8")
135     rgb_image = np.array(cv_image, dtype=np.uint8)
136     if self.image_queue.full():
137         # 如果队列已满, 丢弃最旧的图像(if the queue is full, discard the oldest image)
138         self.image_queue.get()
139     # 将图像放入队列(put the image into the queue)
140     self.image_queue.put(rgb_image)

```

Image node callback function: used to process images and enqueue them.

Move:

```

150 def move(self, *args):
151     if args[0].angular.z == 1:
152         time.sleep(0.2)
153         motor1 = MotorState()
154         motor1.id = 2
155         motor1.rps = 0.1
156         motor2 = MotorState()
157         motor2.id = 4
158         motor2.rps = -1.0
159         msg = MotorsState()
160         msg.data = [motor1, motor2]
161         self.motor_pub.publish(msg)
162         time.sleep(11)
163         motor1 = MotorState()
164         motor1.id = 2
165         motor1.rps = 0.0
166         motor2 = MotorState()
167         motor2.id = 4
168         motor2.rps = 0.0
169         msg = MotorsState()
170         msg.data = [motor1, motor2]
171         self.motor_pub.publish(msg)
172     elif args[0].angular.z == -1:
173         time.sleep(0.2)
174         motor1 = MotorState()
175         motor1.id = 2
176         motor1.rps = 1.0
177         motor2 = MotorState()
178         motor2.id = 4
179         motor2.rps = -0.1
180         msg = MotorsState()
181         msg.data = [motor1, motor2]
182         self.motor_pub.publish(msg)
183         time.sleep(12)
184         motor1 = MotorState()
185         motor1.id = 2
186         motor1.rps = 0.0

```

Movement strategy function that moves the vehicle according to the recognized limb height.

buzzer_warn:

```

201     def buzzer_warn(self):
202         msg = BuzzerState()
203         msg.freq = 1900
204         msg.on_time = 0.2
205         msg.off_time = 0.01
206         msg.repeat = 1
207         self.buzzer_pub.publish(msg)

```

Buzzer control function used for buzzer alarms.

image_proc:

```

209     def image_proc(self, image):
210         image_flip = cv2.flip(cv2.cvtColor(image, cv2.COLOR_RGB2BGR), 1)
211         results = self.body_detector.process(image)
212         if results is not None and results.pose_landmarks is not None:
213             if self.move_finish:
214                 twist = Twist()
215                 landmarks = get_joint_landmarks(image, results.pose_landmarks.landmark)
216                 distance_list = (joint_distance(landmarks))
217
218                 if distance_list[0] < 1:
219                     self.detect_status[0] = 1
220                 if distance_list[1] < 1:
221                     self.detect_status[1] = 1
222                 if 0 < distance_list[2] < 2:
223                     self.detect_status[2] = 1
224                 if 0 < distance_list[3] < 2:
225                     self.detect_status[3] = 1

```

Function for recognizing limbs, which invokes the model to draw key points of the human body based on the recognized information, and moves according to the recognized height.

Main:

```

289     def main(self):
290         while self.running:
291             try:
292                 image = self.image_queue.get(block=True, timeout=1)
293             except queue.Empty:
294                 if not self.running:
295                     break
296             else:
297                 continue
298             try:
299                 result_image = self.image_proc(np.copy(image))
300             except BaseException as e:
301                 self.get_logger().info('\033[1;32m%s\033[0m' % e)
302                 result_image = cv2.flip(cv2.cvtColor(image, cv2.COLOR_RGB2BGR), 1)
303             self.fps.update()
304             result_image = self.fps.show_fps(result_image)
305             cv2.imshow(self.name, result_image)
306             key = cv2.waitKey(1)
307             if key == ord('q') or key == 27: # 按q或者esc退出(press Q or Esc to exit)
308                 self.mecanum_pub.publish(Twist())
309                 self.running = False
310
311         rclpy.shutdown()

```

The main function within the BodyControlNode class, used to input image information into the recognition function and display the live camera feed.